

Flight Management System

Capstone feature list | FastAPI + n8n + Supabase Postgres + Pinecone + RAG + Gmail

This document lists every feature/edge case for the flight management capstone, grouped into 10 domains (56 items total). Architecture: FastAPI is the only write path for live booking and admin operations, and writes directly to Supabase Postgres. n8n reads and writes the same Postgres tables independently for scheduled/automated work — it has no dependency on FastAPI and no shared code contract with it. Each item below is tagged with its owner.

[FastAPI] live, request-triggered, transactional writes

[n8n] scheduled/background, direct-to-Postgres

[Both] shared design decision or coordination point

Admin & Flight Management

FastAPI owns all flight-creation and inventory-defining writes; n8n has no direct role here.

- [FastAPI] Admin endpoint to create a new flight: origin, destination, departure/arrival datetime, flight number
- [FastAPI] Example: create UK → Dubai flight departing 05:00, 100 total seats — 20 First, 30 Business, 50 Economy
- [FastAPI] Validation that seat class totals sum exactly to declared aircraft capacity
- [FastAPI] Reject seat class allocation with negative or zero values, or non-integer seat counts
- [FastAPI] Admin endpoint to edit flight schedule (time/route change) with cascading effect on existing bookings
- [FastAPI] Admin endpoint to cancel a flight entirely, triggering downstream rebooking/refund flow
- [FastAPI] Seat map/layout definition per flight (which physical seats belong to which class)
- [FastAPI] Adjusting seat class allocation after bookings exist — cannot shrink a class below its already-booked count
- [FastAPI] Duplicate flight-number detection for the same day/route
- [FastAPI] Admin role permission tiers (super-admin vs. ops-agent create/edit rights)
- [FastAPI] Audit log of every admin change to a flight (who changed what, when)

Search & Fare Rules

Search and pricing logic must be transactionally consistent with live inventory — FastAPI territory.

- [FastAPI] Search endpoint returning available seats per class for a given route/date
- [FastAPI] Fare class rules: basic economy (no changes, no seat choice) vs. flexible fare
- [FastAPI] Price-hold duration between search results and booking confirmation
- [FastAPI] Multi-leg/connecting itinerary where one leg's fare disappears before the other is booked
- [FastAPI] Currency/locale handling for displayed fares

Seat Holds & Booking

The core overselling-prevention problem — must be atomic, so it lives entirely in FastAPI against Postgres.

- [FastAPI] Temporary seat hold during checkout with expiry if payment isn't completed

- **[FastAPI]** Atomic seat-class decrement to prevent two simultaneous bookings selling the same last seat
- **[FastAPI]** Idempotency key handling for duplicate/retried booking requests
- **[FastAPI]** Explicit overbooking policy per class (allowed with buffer vs. hard never-oversell guarantee)
- **[FastAPI]** Group booking of N seats where only some are available — partial hold, partial fail, or full fail
- **[FastAPI]** Class-specific booking rules (e.g. First/Business allow later cutoff than Economy)

Changes, Cancellations & Refunds

Cancellation/refund decisions are FastAPI; escalation on a stuck refund on a schedule is n8n.

- **[FastAPI]** Cancellation policy branching by fare type (refundable / credit-only / non-refundable)
- **[FastAPI]** Partial cancellation on a multi-passenger booking — proportional refund and re-pricing
- **[FastAPI]** Airline-initiated schedule change — automatic rebooking rule and fare-policy override
- **[FastAPI]** Flight cancellation — refund vs. rebook vs. travel credit, with credit expiration
- **[n8n]** Escalation notification if a refund stays unresolved after N days

Waitlist & Standby

Joining a waitlist is a live booking action (FastAPI); promoting people off it is background polling (n8n).

- **[FastAPI]** Add passenger to waitlist when a class/flight is full
- **[FastAPI]** Waitlist priority rule (loyalty tier vs. booking time vs. fare class)
- **[n8n]** Scheduled job to detect freed-up seats and promote the next waitlisted passenger
- **[n8n]** Notification window before an auto-promoted seat is reassigned to the next person if unclaimed
- **[Both]** Row-locking so a gate-agent action and an n8n promotion run can't assign the same freed seat twice

Scheduled Automations

Time-based, no live user request in the loop — this is n8n's core job.

- **[n8n]** Check-in reminder email, timezone-correct for origin/destination
- **[n8n]** Price-drop alert with de-duplication so it doesn't fire on every micro-fluctuation
- **[n8n]** Daily/weekly ops reporting (load factor, revenue per flight) pulled from Postgres
- **[n8n]** Detection and suppression of reminder workflows for flights already cancelled

Fraud, Policy & RAG Support

Kept entirely in n8n — scheduled jobs reading directly from Postgres and Pinecone, no FastAPI involvement.

- **[n8n]** RAG-drafted answer to policy questions must reflect the booking's actual fare rule, not a generic match
- **[n8n]** Human approval gate before any RAG-drafted answer is sent via Gmail
- **[n8n]** Scheduled bot/mass-booking fraud-scoring job scanning recent bookings
- **[n8n]** Scheduled ingestion pipeline embedding updated policy docs into Pinecone
- **[n8n]** Batch review job scanning historical bookings for fraud patterns missed in real time

Approval & Autonomy Boundaries

Applies across both systems — the policy itself is a design artifact, not code.

- **[Both]** Explicit list of actions auto-approved (reminders, standard in-policy refunds)
- **[Both]** Explicit list of actions requiring human sign-off (schedule-change compensation, denied-boarding compensation)
- **[Both]** Audit trail sufficient to justify an automated decision to a regulator or complaint review

Shared Database Architecture (FastAPI + n8n dual-writer)

n8n reads/writes Postgres directly with no FastAPI contract — these rules keep the two systems from corrupting each other's data.

- **[Both]** Table-ownership rules: which system is the sole writer of which tables
- **[Both]** Postgres-level invariants (CHECK constraints, foreign keys, enums) since n8n bypasses FastAPI validation
- **[n8n]** Change-detection mechanism for n8n: polling vs. LISTEN/NOTIFY vs. a status/flag column, with justification
- **[n8n]** Row-locking strategy (e.g. SELECT ... FOR UPDATE SKIP LOCKED) to prevent double-processing in polling workflows
- **[Both]** Reconciliation/audit job to catch inconsistent states from a failed or partial n8n write
- **[Both]** Explicit conflict-resolution rule for concurrent writes (e.g. FastAPI cancellation vs. n8n waitlist promotion on the same seat)

Core Infrastructure

The connective tissue across the whole stack.

- **[FastAPI]** Typed request/response schemas and validation for every booking-affecting endpoint
- **[FastAPI]** Idempotency header/key contract for all write endpoints
- **[Both]** Supabase Postgres as the single ledger of truth for flights, seats, and bookings
- **[n8n]** Pinecone index maintenance job for policy docs and fraud-pattern embeddings
- **[n8n]** Gmail node integration for all customer-facing scheduled notifications
- **[FastAPI]** Gmail send for transactional, request-triggered emails (booking confirmation, cancellation receipt)

Generated feature/edge-case checklist with FastAPI vs. n8n ownership — hackathon-style capstone testing architectural and business-rule judgment, not prompt-to-JSON generation.